

OCamlで作ったアプリを 実運用している

Yuki Tajiri @Imdexpr

2026-07

今回の話の対象者

L = **Layer** (FPL との関わり方＝立ち位置)。難易度ではないので、気になる Layer をつまみ食いで OK

L1 興味がある

- FPL やマイナーな言語に興味がある
- それらを採用した時にどうなるのか知りたい

L2 採用したい

- FPL やマイナーな言語を採用したい
- それらを採用するにはどうしたらいいのか知りたい

L3 もう採用している

- 既に何らかの FPL を採用していて、OCaml のことが知りたい

1. 自己紹介

2. 何のアプリを作ってるの？

3. 実際どうなの？ L1

4. なぜ OCaml (+ReScript) で作ってるの？ L2

5. もっと具体的な話 L2 L3

自己紹介 (1/2)

- 田尻裕喜
- 関数型まつり 2025/2026 スタッフ
 - 別部屋でワークショップやってます
- λ Kansai 運営
 - 9 月頃に初心者向けの会を開催予定
- OCaml Meeting 2026 in Tokyo 言い出しっぺ
 - 参加者もう埋まってしまいました

自己紹介 (2/2)

- エムスリー 基盤開発チーム エンジニア
 - 認証認可サービス
 - ポイント関連サービス
 - メルマガ配信サービス
 - API Gateway
 - ファイアウォール
 - その他色々

1. 自己紹介
2. 何のアプリを作ってるの？
3. 実際どうなの？ L1
4. なぜ OCaml (+ReScript) で作ってるの？ L2
5. もっと具体的な話 L2 L3

- 認証認可サービス (Java, Ruby)
- ポイント関連サービス (Java, Kotlin)
- メルマガ配信サービス (Ruby)
- API Gateway (Scala)
- ファイアウォール (Scala)
- その他色々 (Go, JS, TS, **OCaml**)

カジュアル面談や一部面接の日程調整アプリを作っています

(ここにQAコード)

(カジュアル面談実際に申し込んでもらってもいいですよ)

- カジュアル面談や一部面接の日程調整アプリを作っています
- なぜ？
 - 日程調整パズルはつらい
 - 既存アプリだと社内の細かな事情に対応できない
 - 採用を頑張るためにはできる限り「直近で」「高速に」予定を合わせたい
- 実際の採用活動で使うので、普通に壊れると困る
 - カレンダー、Slack、メール、LLM など外部サービスとも連携
 - public に公開されている

Languages



- OCaml 52.5%
- ReScript 37.7%
- HCL 6.3%
- Shell 1.2%
- TypeScript 0.9%
- Dockerfile 0.6%
- Other 0.8%

実際に使われている言語

1. 自己紹介
2. 何のアプリを作ってるの？
3. 実際どうなの？ L1
4. なぜ OCaml (+ReScript) で作ってるの？ L2
5. もっと具体的な話 L2 L3

結論：困ってない

- 「OCamlだから」「マイナーだから」といった問題はあまりない
- 逆に言えば、他の言語でも起きうる問題はある
- つまり、「普通の」技術選定と同じ

なぜ困らないのか

- 今の OCaml は Web アプリに必要な土台が揃っている
- 型・小さい言語仕様・後方互換性で保守しやすい
- 足りない部分は小さく自作でき、AI も補助になる

よくある問題

- 技術的な問題
 - ライブラリ・標準ライブラリが貧弱
 - エコシステムが揃ってない
- 人の問題
 - 困った時に助けてくれる人が少ない
 - 人の採用・オンボードが難しい

技術的な問題 (1/2)

- Claude / Devin / Copilot は OCaml でも普通に書いてくれる
 - ここ数年で出たライブラリでも特に問題なし
- ライブラリが足りない問題も、小さい境界なら自作しやすい
 - 自分に必要な範囲だけを実装する
 - AI はその実装とレビューの補助としてかなり効く
- HTTP / JSON / OAuth / JWT / TLS / ログ とかは揃ってる
- 足りないライブラリ
 - AWS / Google / Slack / ...
 - 私の手元にはあるので出せるものから OSS にしていきます
 - AWS なんかは良さげなライブラリが出始めてもいる

技術的な問題 (2/2)

- エコシステム周りも整備され始めている
 - まだまだこれからな部分はある
- マイナー言語を採用するのであれば、ここと向き合う覚悟は必要
- ただ、ここも AI がかなり助けてくれるので意外と問題はないかも？

人の問題

- 開発は一人でやってます
 - 小さく始めているので、今のところ問題になっていない
 - AI による実装補助も効いている
- OCaml に関して言えば **L3**
 - OCaml Meeting、日本語 Discord など日本コミュニティが今は熱い
 - どちらも私がいるので困ったことがあれば、一緒に困りましょう
 - イベントの二ヶ月前に 50 人集まるポテンシャルがあります

1. 自己紹介
2. 何のアプリを作ってるの？
3. 実際どうなの？ L1
4. なぜ OCaml (+ReScript) で作ってるの？ L2
5. もっと具体的な話 L2 L3

好きだから

好きだから

...で済ませると登壇にならないので続きます

「好きだから」では採用できない？

技術選定のためには、

- 課題の要件を満たせること
- 継続して運用できること

「好きだから」では採用できない？

技術選定のためには、

- 課題の要件を満たせること
- 継続して運用できること

→ この辺りの説明のためには、ちゃんと使っておくことが必要

採用前に答えたい問い

例えば、

- 言語バージョンはどれがいいの？
- ビルドシステムは何があるの？
- LSP や formatter などのツールチェインはある？
- CI / CD は問題なく構築できる？
- ライブラリの lock やアップデートの仕組みは？
- 開発環境の体験は良いの？
- 実行環境の問題はないか？

くらの質問の答えは意識しておきたい

OCaml/ReScript の場合

- 言語バージョンはどれがいいの？ → 一旦最新に追従
- ビルドシステムは何があるの？ → dune/rescript
- LSP や formatter などのツールチェインはある？ → 十分動作する
- CI / CD は問題なく構築できる？ → 基本は大丈夫
- ライブラリの lock やアップデートの仕組みは？ → dune pkg management
- 開発環境の体験は良いの？ → 別に悪くない
- 実行環境の問題はないか？ → ネイティブ動作でマルチコア対応もある

最低限揃ったとして、どうやって採用するの(したの)?

採用できた色々な要素

- 小さいところから始める
- 自身のブランディングと周囲の理解
- AI 時代だから

小さいところから始める

- 「社内用だし、好きな技術でもいいですよ？」
- 「小さいアプリだし、前に使ったから良いですよ？」
- 「稼働実績あるから、public に公開しても良いですよ？」

→ 新しい技術の選定はいつだってこんな経路が理想的かも

自身のブランディングと周囲の理解

- 普段から「この技術が好き！」を押し出す
 - times でずっとその話をする
 - 社内の勉強会・テックブログでずっとその話をする
 - 社外の活動でもずっとその話をする
- 周囲に「あの人はこれさせとけば良いんだな」と思ってもらう
 - いっそ自分から「これあれば十分です！」と言っちゃう

→ 3ヶ月もやってれば「〇〇の人」になり、周囲の理解が得られる

マイナーな言語ほどブランディングはやりやすい

AI 時代だから

- LLM はかなり成長した
 - 多少知らなくても一緒に勉強していける環境になった
- 少なくとも Spring(Java), Rails(Ruby) に比べて、明確な差はない
 - どの技術で作っても、十分書いてくれるし、それなりに間違える
 - ハルシネーションの「種類」はちょっと違うかも
- 詰まった時に捨てる判断も簡単になった

→ 結果として、言語選択の自由度が上がった

**何使っても良いなら
人間が気分良い言語が良くないですか？**

好きだから採用してもいい時代

1. 自己紹介
2. 何のアプリを作ってるの？
3. 実際どうなの？ L1
4. なぜ OCaml (+ReScript) で作ってるの？ L2
5. もっと具体的な話 L2 L3

今の OCaml

- OCaml 5.0 で multicore / effect handlers が入った
- 既存の強みである型・パターンマッチ・後方互換性はそのまま
- “古い研究言語”ではなく、今も更新されている実用言語

何が良いの OCaml

- 強めの後方互換性
- シンプルな仕様
- 強力な型推論
- Effect でコンテキスト分離
- Polymorphic variant でエラー表現

Effect でコンテキスト分離

```
(* 副作用は effect として「宣言」するだけ。実装は持たない *)
type _ Effect.t += Now : float Effect.t
type _ Effect.t += Find_user : id -> user Effect.t

(* domain: ロジックは I/O 実装を持たず、必要な effect を perform する *)
let greet id =
  let user = Effect.perform (Find_user id) in
  Printf.sprintf "%s さん、こんにちは (%f)" user.name (Effect.perform Now)

(* adapter: handler で実装を外から注入する = DI *)
let run ~(find_user : id -> user) ~(now : unit -> float) main =
  match main () with
  | result -> result
  | effect Now, k -> Effect.Deep.continue k (now ())
  | effect (Find_user i), k -> Effect.Deep.continue k (find_user i)

(* test では同じ greet にモックを注入できる *)
let () = run ~find_user:mock_find ~now:(fun () -> 0.0) (fun () -> greet id)
```

Polymorphic variant でエラー表現

```
open Result.Syntax
```

```
(* domain: 業務ルールのエラーだけを返す。DB の都合は知らない *)  
let reserve : calendar -> slot -> (booking, [> `Already_booked ]) result = ...
```

```
(* port: usecase が依存する境界。実装は adapter 側にある *)  
let find_user : id -> (user, [> `User_not_found | `User_store_error ]) result = ...
```

```
(* usecase: port と domain のエラーが推論で合流していく *)  
let book req =  
  let* user = find_user req.user_id in  
  let* booking = reserve user.calendar req.slot in  
  Ok booking
```

```
(* adapter: 境界で全エラーを網羅して HTTP に変換。ケース漏れはコンパイラが検出してくれる *)  
let to_http = function  
  | `User_not_found -> 404  
  | `Already_booked -> 409  
  | `User_store_error -> 503
```

何が良いの ReScript

- OCaml から派生した AltJS
- 端的にいうと「型安全でパターンマッチがある TypeScript」
- OCaml からは離れようとしている...?
 - 過去に存在した integration がなくなっている
- 個人的には、適材適所で使いたいののでちょうど良い

→ ReScript については「フロントエンドカンファレンス関西」などで詳しく話す予定です

内製しているもの

- OCaml
 - AWS クライアント
 - Google カレンダー API クライアント
 - Slack クライアント
 - LLM オーケストレーター
- ReScript
 - State monad
 - jotai like State store
- その他
 - OCaml コード → openapi.json ジェネレータ
 - jsonschema → ReScript コードジェネレータ

今のところ sigv4.ml だけ公開しています。

<https://github.com/lmdexpr/sigv4.ml>

その後も気が向いた順に公開するかも.....？

まとめ

- OCaml 運用はやっていける
- **AI 時代は好きな言語を採用できる絶好の機会**
- 今熱いので OCaml をみんなで書こう！